

Operating Systems and Concurrency

Processes 2, Scheduling

COMP2007

Dan Marsden

(Chao Chen)

{Chao.Chen, Dan.Marsden}@Nottingham.ac.uk

University Of Nottingham

United Kingdom

2026

Recap

Last Lecture

- Processes have **control structures** associated with them, the **process control blocks** and **process tables**.
- Processes can have different **states** and the kernel triggers **transitions** between these states.
- The operating system maintains multiple **process queues**.
- The operating system **manages processes** on the user's behalf.

Goals for Today

Overview

- Introduction to **process scheduling**
- Types of **process schedulers**
- **Evaluation criteria** for scheduling algorithms
- Typical **process scheduling algorithms**

Process Scheduling

Context

- The OS is responsible for **managing** and **scheduling processes**
 - Decide when to **admit** processes to the system (new → ready)
 - Decide which process to **run** next (ready → run)
 - Decide when and which processes to **interrupt** (running → ready)
- It relies on the **scheduler (dispatcher)** to decide which process to run next, which uses a **scheduling algorithm** to do so
- The type of algorithm used by the scheduler is influenced by the **type of operating system** (e.g., real time vs. batch)

Process Schedulers

Classification by Time Horizon

- **Long term:** applies to **new processes** and controls the degree of multiprogramming by deciding which processes to admit to the system when
 - A good **mix** of **CPU** and **I/O bound processes** is favourable to keep all resources as busy as possible
 - **Usually absent** in popular modern OS
- **Medium term:** controls swapping and the degree of multi-programming
- **Short term:** decide which process to run next
 - Manages the **ready queue**
 - Invoked very **frequently**, hence must be **fast**
 - Usually called in response to **clock interrupts**, **I/O interrupts**, or **blocking system calls**

Process Schedulers

Classification by Time Horizon

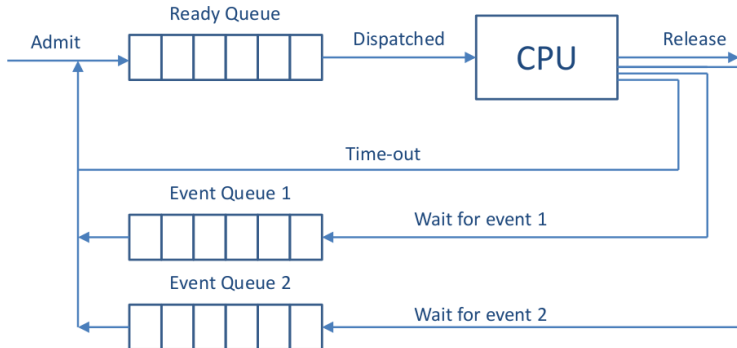


Figure: Queues in OS

Process Schedulers

Classification by Approach

- **Non-preemptive:** processes are only interrupted **voluntarily** (e.g., I/O operation or “nice” system call – `yield()`)
 - Windows 3.1 and DOS were non-preemptive
- **Preemptive:** processes can be **interrupted forcefully** or **voluntarily**
 - Typically **driven by interrupts from a system clock**.
 - Requires additional context switches which generate **overhead**, too many of them should be avoided (recall last lecture)
 - Prevents processes from **monopolising the CPU**
 - **Most popular** modern operating systems are preemptive

Performance Assessment

Criteria

- **User oriented criteria:**

- **Response time:** minimise the time between creating the job and its first execution
- **Turnaround time:** minimise the time between job creation and completion
- **Predictability:** minimise the variance in processing times

- **System oriented criteria:**

- **Throughput:** maximise the number of jobs processed per hour
- **Fairness:**
 - Are processing power/waiting time equally distributed?
 - Are some processes kept waiting excessively long (**starvation**)

- Evaluation criteria can be **conflicting**, i.e., **improving the response time** may require **more context switches**, and hence **worsen the throughput** and **increase the turn around time**

Scheduling Algorithms

Overview

- **Algorithms** considered:
 - 1 First Come First Served (**FCFS**)/ First In First Out (**FIFO**)
 - 2 **Shortest job first**
 - 3 **Round robin**
 - 4 **Priority queues**
- Performance measures used:
 - **Average response time**: the average of the time taken for all the processes to start
 - **Average turnaround time**: the average time taken for all the processes to finish

Scheduling Algorithms

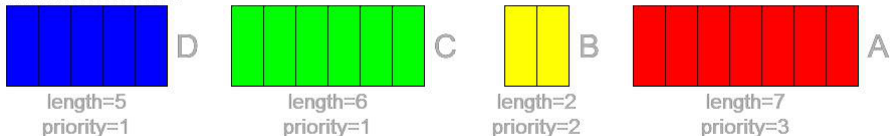
First Come First Served

- Concept: a **non-preemptive algorithm** that operates as a **strict queueing mechanism** and schedules the processes in the same order that they were added to the queue
- Advantages: **positional fairness** and easy to implement
- Disadvantages:
 - **Favours long processes** over short ones (think of the supermarket checkout!)
 - Could **compromise resource utilisation**, i.e., CPU vs. I/O devices

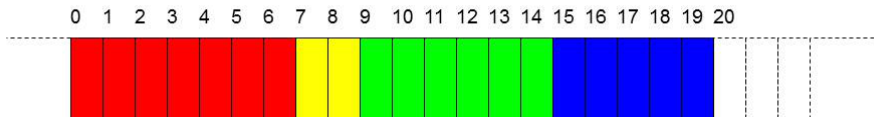
Scheduling Algorithms

First Come First Served

Process Queue



CPU



- Average response time = $\frac{0+7+9+15}{4} = \frac{31}{4} = 7.75$
- Average turn around time = $\frac{7+9+15+20}{4} = \frac{51}{4} = 12.75$

Scheduling Algorithms

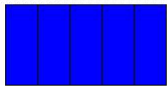
Shortest Job First

- Concept: A **non-preemptive algorithm** that starts processes in order of **ascending processing time** using a provided/known estimate of the processing
- Advantages: always result in the **optimal turn around time**
- Disadvantages:
 - **Starvation** might occur
 - **Fairness** and **predictability** are compromised
 - **Processing times have to be known** beforehand

Scheduling Algorithms

Shortest Job First

Process Queue



D

length=5
priority=1



C

length=6
priority=1



B

length=2
priority=2



A

length=7
priority=3

CPU

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20



- Average response time = $\frac{0+2+7+13}{4} = \frac{22}{4} = 5.5$
- Average turn around time = $\frac{2+7+13+20}{4} = \frac{42}{4} = 10.5$

Scheduling Algorithms

Shortest Job First

Why does shortest job first give optimal turn around times?

Scheduling Algorithms

Shortest Job First

Why does shortest job first give optimal turn around times?

- Assume 4 processes, P_1 , P_2 , P_3 and P_4 .

Scheduling Algorithms

Shortest Job First

Why does shortest job first give optimal turn around times?

- Assume 4 processes, P_1, P_2, P_3 and P_4 .
- Assume the processes are run in order, and the i^{th} process requires T_i milliseconds.

Scheduling Algorithms

Shortest Job First

Why does shortest job first give optimal turn around times?

- Assume 4 processes, P_1, P_2, P_3 and P_4 .
- Assume the processes are run in order, and the i^{th} process requires T_i milliseconds.
- Let S_i be the turnaround time of the i^{th} process.

Scheduling Algorithms

Shortest Job First

Why does shortest job first give optimal turn around times?

- Assume 4 processes, P_1, P_2, P_3 and P_4 .
- Assume the processes are run in order, and the i^{th} process requires T_i milliseconds.
- Let S_i be the turnaround time of the i^{th} process.
- Then:

$$S_1 = T_1 \quad S_2 = T_1 + T_2 \quad S_3 = T_1 + T_2 + T_3 \quad S_4 = T_1 + T_2 + T_3 + T_4$$

Scheduling Algorithms

Shortest Job First

Why does shortest job first give optimal turn around times?

- Assume 4 processes, P_1, P_2, P_3 and P_4 .
- Assume the processes are run in order, and the i^{th} process requires T_i milliseconds.
- Let S_i be the turnaround time of the i^{th} process.
- Then:

$$S_1 = T_1 \quad S_2 = T_1 + T_2 \quad S_3 = T_1 + T_2 + T_3 \quad S_4 = T_1 + T_2 + T_3 + T_4$$

- Therefore the average turnaround time is:

$$\frac{4 \times T_1 + 3 \times T_2 + 2 \times T_3 + 1 \times T_4}{4}$$

Scheduling Algorithms

Round Robin

- Concept: a **preemptive version of FCFS** that forces **context switches** at **periodic intervals** or **time slices**
 - Processes **run in the order that they were added** to the queue
 - Processes are forcefully **interrupted by the timer**
- Advantages:
 - Improved **response time**
 - Effective for general purpose **interactive/time sharing systems**
- Disadvantages:
 - Increased **context switching** and thus overhead
 - **Favours CPU bound processes** (which usually run long) over I/O processes (which do not run long)
 - Can be prevented by working with multiple queues?
 - Can **reduce to FCFS**

Scheduling Algorithms

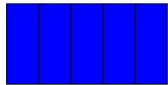
Round Robin

- The **length** of the **time slice** must be carefully considered!
- For instance, assuming a **multi-programming system** with **preemptive scheduling** and a **context switch time** of 1ms:
 - E.g., a **good (low) response time** is achieved with a **small time slice** (e.g. 1ms) $\Rightarrow$ low throughput
 - E.g., a **high throughput** is achieved with a **large time slice** (e.g. 1000ms) $\Rightarrow$ high response time
- If a time slice is only **used partially**, the next process **starts immediately**

Scheduling Algorithms

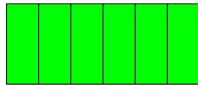
Round Robin

Process Queue



length=5
priority=1

D



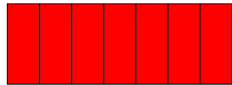
length=6
priority=1

C



length=2
priority=2

B

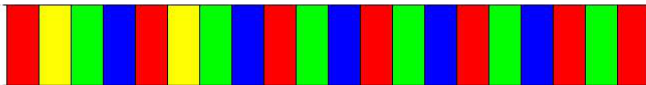


length=7
priority=3

A

CPU

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20



- Average response time = $\frac{0+1+2+3}{4} = \frac{6}{4} = 1.5$
- Average turn around time = $\frac{6+17+19+20}{4} = \frac{62}{4} = 15.5$

Scheduling Algorithms

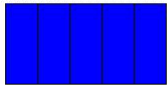
Priority Queues

- Concept: A **preemptive algorithm** that schedules processes by priority (high $\rightarrow$ low)
 - A **round robin** is used **within the same priority levels**
 - The process priority is saved in the **process control block**
- Advantages: can **prioritise I/O bound jobs**
- Disadvantages: low priority processes may suffer from **starvation** (when priorities are static)

Scheduling Algorithms

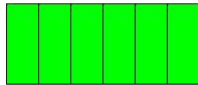
Priority Queues

Process Queue



length=5
priority=1

D



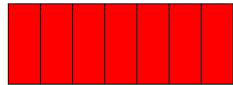
length=6
priority=1

C



length=2
priority=2

B

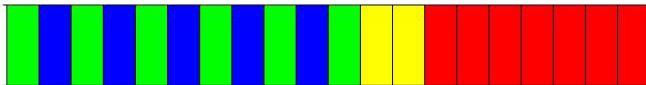


length=7
priority=3

A

CPU

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20



- Average response time = $\frac{0+1+11+13}{4} = \frac{25}{4} = 6.25$
- Average turn around time = $\frac{10+11+13+20}{4} = \frac{54}{4} = 13.5$

Scheduling Algorithms

Priority Queues

- Give the **order in which the processes are scheduled** when using **priority queues**, together with the **times at which they will start, end, and are interrupted** (all processes are available at the time of scheduling)
- You can assume a time slice of 15 milliseconds
- Calculate the **average response and turn around time**

	FCFS Position	CPU burst time	Priority
Process A	1	67	1 (high)
Process B	2	37	1 (high)
Process C	3	14	2 (low)
Process D	4	16	2 (low)

Scheduling Algorithms

Priority Queues

- Solution:
 - Sequence: $A(15) \Rightarrow B(15) \Rightarrow A(15) \Rightarrow B(15) \Rightarrow A(15) \Rightarrow B(7) \Rightarrow A(15) \Rightarrow A(7) \Rightarrow C(14) \Rightarrow D(15) \Rightarrow D(1)$
 - Average response time = $(0 + 15 + 104 + 118) / 4$
 - Average turnaround time = $(82 + 104 + 118 + 134) / 4$
- Note: we ignore **context switch time**

Test your understanding

- On a non-preemptive operating systems, what sort of things can a process do without potentially “volunteering” to cede the CPU to another process.
- Using the non-preemptive shortest job first scheduler, does the shortest job run on the CPU until it is completed?
- On a non-preemptive operating system, how much harm can a single user process do?